Governed Steel Thread

A Software Development Model for Agentic Payment Systems

Project: Agentic UPI Commerce Bridge — AP2/UCP × Razorpay UPI Autopay
Document Version: 1.0
Date: 2026-08-29
Classification: Buildathon Engineering Philosophy / Team Charter
Derived From: Steel Thread (DoD systems engineering), adapted for security-governed AI-to-AI commerce

---

1. Why a New Model?

Traditional software development philosophies fail agentic payment systems for three reasons:

1. **Agile/Scrum** optimizes for feature velocity. In payments, velocity without invariant discipline produces exploits, not value.
2. **Waterfall** assumes requirements freeze early. Agentic commerce requirements evolve as protocol specs (AP2, UCP, A2A) are themselves moving targets.
3. **Lean Startup** treats the MVP as a learning vehicle. A payment MVP that "learns" it leaked a signing key is a criminal liability, not a lesson.

The missing ingredient: A model that treats *security invariants* as load-bearing structural members of the thread itself — not as QA gates applied after construction.

Governed Steel Thread is that model.

---

2. Core Definition

> **Governed Steel Thread** is the practice of building a minimal, end-to-end functional path through a system where every segment of the thread is simultaneously (a) user-demo-able, (b) security-invariant-enforced, and (c) architecturally extensible. No segment may be declared "complete" if its invariants are unverified, regardless of feature completeness.

2.1 The Three Axes

Every thread is evaluated on three axes. A thread is **steel** only when all three are satisfied.

Axis	Question	Non-Steel Indicator
Functional	Does money move from intent to settlement?	"The guardrail compiles but isn' t wired to the vault yet."
Governed	Can the LLM cause an unauthorized debit?	"We' ll add the policy engine in Thread 2."
Observable	Can you reconstruct the full decision path in < 30s?	"Logging is a nice-to-have for the demo."

2.2 The Invariant-First Rule

> **Rule 0:** An invariant (INV-001 through INV-010) is not a test case. It is a structural dependency. If a thread cannot yet verify an invariant, that section of the architecture is treated as **unresolved load** — like a bridge span without tension cables — and the thread does not advance past it.

This is the critical divergence from classical Steel Thread, where "working end-to-end" is sufficient. In governed systems, "working" includes "cannot be made to work unsafely."

---

3. Philosophical Foundations

> **"Working System First, Perfect System Later."**
> The goal is a working prototype by 5 September. Production-grade concerns (HSM, Kafka, service mesh, TLA+ verification) are documented as [Production] but not implemented. This is explicitly stated in the SRS and will be acknowledged in the pitch.

3.1 The Deterministic Sandwich as Architectural Spine

The system's architecture is not a layered cake where the LLM sits at the top with "guardrails" sprinkled below. It is a **sandwich**:

```
[Deterministic Input] → [Probabilistic Core] → [Deterministic Output]
```

The authority chain is:

```
LLM reasoning
    ↓
Structured proposal
    ↓
Deterministic validation
    ↓
Policy / grounding / authorization checks
    ↓
Cryptographic authorization
    ↓
Settlement
    ↓
Audit
```

The LLM is therefore a proposal generator, not an authorization mechanism.

- The **bread** is steel. The **filling** is probabilistic.
- You do not build the filling first and hope to add bread later. You build the bread, verify it is airtight, then insert the filling.
- For this project, Thread 0 is the bread. Threads 1+ add filling and condiments.

3.2 Fail-Closed as Default Posture

> **Rule 1:** In the absence of a positive signal, the system does not "degrade gracefully." It halts. Graceful degradation is a UX concept. Fail-closed is a safety concept. This project uses the latter.

Every component must answer: *"If I am unreachable, misconfigured, or compromised, what is the safest thing the caller can do?"* The answer is always: **reject, escalate, or abort** — never pass-through, default-allow, or best-effort.

3.3 Explainability as Non-Functional Requirement

> **Rule 2:** A payment action without an audit trail is indistinguishable from a bug or an attack. Therefore, observability is not DevOps overhead. It is a correctness requirement.

The buildathon judges will not accept "it works" as evidence. They require explainability, boundedness, and gating. The development model must therefore produce **evidence** at every thread boundary, not just functionality.

3.4 Zero Trust in the Probabilistic Core

> **Rule 3:** The LLM is treated as an untrusted, potentially adversarial input source — even when it is your own code calling it. The Guardrail Shell is not a "check." It is the **only authorization boundary**.

This means:

- No feature is designed assuming the LLM "will probably do the right thing."
- Every LLM output is treated as untrusted data until validated by deterministic code.
- The LLM container has no network path to keys, credentials, or payment APIs, by design, not by configuration.

---

4. Thread Definitions for This Project

4.1 Thread 0 — The Deterministic Spine

Goal: Prove that money cannot move without passing through every layer of the sandwich, and that failure is graceful, explainable, and audited.

Scope (8-day buildathon):

- One human intent → one compiled constraint
- One LLM sub-agent → one ProposalObject
- Guardrail Shell: schema, policy, grounding, confidence (with HITL escalation path)
- Mandate Vault: real ES256 JWS signing
- Payment Adapter: Razorpay Test Mode e-mandate lifecycle
- Ledger: append-only, hash-chained, queryable
- One graceful failure recorded live (revocation racing debit)

Stubbed:

- Parallel negotiation (1 merchant only)
- OpenTelemetry (structured JSON logging)
- 500-vector injection suite (20 hand-crafted vectors)
- Full UCP discovery layer (manual manifest)

Invariant Verification Gates (must pass before Thread 0 is steel):

- INV-001: LLM container cannot read Vault secret from filesystem, network, or env
- INV-002: Guardrail Shell is the only path to Vault/Adapter (code review + integration test)
- INV-003: Duplicate `(mandate_id, idempotency_key)` is rejected at database level
- INV-004: `SELECT ... FOR UPDATE` ensures revocation wins race (concurrent load test)
- INV-005: Audit record timestamp execution timestamp in all test cases
- INV-006: Cross-component DELETE on audit table returns permission denied
- INV-007: Random unsupported constraint → never silent drop (property test)
- INV-008: Malicious manifest → Guardrail blocks proposal (injection test)
- INV-009: Altered canonical byte → signature verification rejects (crypto negative test)
- INV-010: Policy Engine rejects proposal exceeding hard-coded max_spend regardless of LLM reasoning

4.2 Thread 1 — Probabilistic Negotiation

Goal: Add the LLM's value (parallel discovery, ranking, negotiation) without weakening the spine.

Scope:

- Up to 3 parallel sub-agents (demo), architectured for 10
- MCP A2A adapter shim
- Confidence formula calibration against golden dataset
- Monotonicity check and circuit breaker
- Cart locking with Redis

Constraint: Thread 1 must not modify Thread 0's trust boundaries. The Guardrail Shell interface contract remains identical.

4.3 Thread 2 — Scale & Hardening

Goal: Production-readiness without architectural fracture.

Scope (post-buildathon):

- gRPC/Protobuf migration (OpenAPI spec as SSOT)
- HSM/KMS-backed keys
- Kafka outbox + Redis Streams
- Full OpenTelemetry stack

- 500-vector injection benchmark
- External red-team engagement
- TLA+ formal verification of UCP state machine

4.4 Risk-First Development Order

Recommended ordering:

1. Unauthorized payment
2. Signing-key compromise or signing-oracle behavior
3. Mandate replay and race conditions
4. Constraint bypass
5. Prompt injection leading to privileged action
6. Protocol incompatibility
7. Ambiguous payment/reconciliation states
8. Data/audit integrity
9. Availability/performance
10. Feature richness and negotiation sophistication

The most important question is not "Can the agent complete a purchase?" but:

> Can the system prevent an unauthorized purchase even when individual components behave maliciously, incorrectly, or unexpectedly?

---

5. Execution Framework

5.1 The Daily Steel Check

At the end of each day, the developer must answer three questions:

1. **What moved today?** (Functional axis)
2. **What is now impossible?** (Governed axis — which attack vector did we close?)
3. **What can I prove?** (Observable axis — which trace/audit/log demonstrates it?)

If any axis has no answer, the next day's priority is that axis, not new features.

5.2 Scope Negotiation Protocol

When time pressure threatens a deadline, the default response is **stub, don't skip**.

Threat	Wrong Response	Governed Response
"We don' t have time for the full Guardrail Shell."	Ship without grounding.	Stub the LLM layer; wire the Guardrail to reject everything and escalate to HITL. The invariant holds.
"OpenTelemetry is too complex for the demo."	Ship without traces.	Emit structured JSON logs to stdout with `mandate_id` and `transaction_id`. The observable axis holds.
"We can' t get real Shopify data in time."	Skip the commerce adapter.	Hand-craft one UCP manifest file. The functional axis holds for demo scope.

Table 2 – continued

Threat	Wrong Response	Governed Response
"The confidence formula isn' t calibrated."	Guess a threshold.	Set threshold to 1.0 (always escalate). The governed axis holds; calibration becomes Thread 1.

5.3 The Demo as Acceptance Test

The buildathon demo is not a marketing exercise. It is the **final integration test** for Thread 0.

Demo script as test case:

```
Given a human has compiled a purchase intent with max_spend = 5000 INR
And a merchant has offered a product at 4999 INR
When the LLM proposes the merchant
And the Guardrail computes confidence C >= 0.85
And the Mandate Vault signs the Payment Mandate
And the Payment Adapter executes against Razorpay Test Mode
Then the ledger records a SETTLED event with matching constraint_hash
And the Jaeger trace shows the full decision path
And the JSONL export is tailable live

Given the same scenario
But the user revokes the mandate during debit execution
Then the atomic lock ensures the debit is rejected
And the response is 403 MANDATE_REVOKED
And the audit log shows REVOKED before DEBIT_ATTEMPT
```

If the demo cannot be written as a passing Gherkin spec, the thread is not steel.

5.4 Security-Gated Definition of Done

Every feature that touches authorization, cryptography, external content, identity, payment, or audit shall satisfy:

Functional

- Happy path works
- Required interface contract is implemented
- Expected failure states are implemented

Security

- Trust boundary is explicit
- Secrets/keys are correctly scoped
- Unauthorized paths are blocked

Invariants

- At least one explicit invariant is mapped to the implementation

Testing

- Positive test + negative test + boundary/edge case
- Concurrency test where state races are possible
- Security regression test where an attack path is relevant

Evidence

- Test result, trace, audit event, state transition, or cryptographic verification result that proves the property

---

6. Comparison with Other Models

Model	Fit for Agentic Payments	Why Governed Steel Thread Differs
Waterfall	❌ Too rigid; cannot adapt to evolving protocol specs	GST is iterative within threads, not sequential across phases
Agile/ Scrum	⚠️ Velocity-obsessed; sprints encourage feature completion over invariant verification	GST declares a sprint "incomplete" if invariants are unverified, regardless of story points
Lean Startup	⚠️ "Build-Measure-Learn" treats safety as a hypothesis	GST treats safety as a prerequisite for building
Classical Steel Thread	✅ Close, but insufficient	GST adds the **Governed** axis —invariants are structural, not tested-in-later
Shape Up	⚠️ Fixed time, variable scope —fits hackathons	GST adds the rule that scope can only be cut from the **probabilistic** layer, never the **deterministic** layer
Safety-Critical Spiral	✅ Appropriate rigor, but too heavy for 8 days	GST is Spiral compressed: risk analysis happens per-thread, not per-cycle

---

7. Anti-Patterns

These are explicitly prohibited under this model:

1. **"We'll add the guardrail in v2."** — The guardrail is the spine. Without it, there is no v1.
2. **"The LLM is smart enough to not need hard limits."** — INV-010 exists because the LLM is not trusted with exposure bounds, ever.
3. **"Let's get it working, then we'll audit it."** — Audit is not a coating. It is a structural member. Thread 0 includes the ledger.
4. **"We can use the same key for signing and identity to save time."** — FR-MV-004 (key partitioning) is not optimization. It is blast-radius containment.
5. **"Test mode doesn't need mTLS."** — SEC-NET-003 applies to Test Mode. The demo environment models production trust boundaries.
6. **Feature-first development:** Do not build sophisticated negotiation before proving the authorization boundary.
7. **Security-by-prompt:** Do not assume that a model instruction such as "never spend more than X" constitutes an enforceable spending limit.

8. **Silent protocol degradation:** Do not silently discard a constraint that the settlement rail cannot enforce.
9. **Happy-path-only testing:** Do not declare payment functionality complete until replay, invalid authorization, dependency failures, and ambiguous provider responses are tested.
10. **Security claims without evidence:** Do not describe a property as "secure" or "isolated" unless the architecture and tests demonstrate the claim within the stated threat model.

---

8. Team Charter

By adopting this model, the team agrees:

- **I will not declare a component "done" until its invariants are demonstrably held.**
- **I will stub probabilistic features before I skip deterministic validation.**
- **I will treat every LLM output as potentially malicious until the Guardrail proves otherwise.**
- **I will write the demo script as a Gherkin spec on day 1, and code toward making it pass.**
- **I will ask "what is now impossible?" at the end of every day, not just "what works?"**

---

9. References

- SRS v1.1 — Agentic UPI Commerce Bridge (consolidated from six role-perspective documents)
- SDD v1.0 — System Design Document (module contracts, database schemas, deployment topology)
- Steel Thread Model — DoD systems engineering practice; adapted from Coplien & Bjørnvig, *Lean Architecture*
- AP2 Specification — Google Agent Payments Protocol (v0.1, January 2026)
- UCP Specification — Universal Commerce Protocol (Google + Shopify, NRF Jan 2026)
- A2A Protocol — Agent-to-Agent Protocol (Google → Linux Foundation)
- RFC 8785 — JSON Canonicalization Scheme (JCS)
- RFC 7515 — JSON Web Signature (JWS)

---

10. Final Guiding Principle

Build the narrowest complete system that makes the critical security properties real. Prove those properties with deterministic tests. Then expand the system without weakening the boundaries that made the first thread trustworthy.

Or, operationally:

```
THIN THREAD
    ↓
REAL SECURITY BOUNDARIES
    ↓
DETERMINISTIC ENFORCEMENT
    ↓
FAILURE TESTS
```

```
    ↓
INVARIANT PROOF
    ↓
EVIDENCE
    ↓
EXPAND
```

End of Document